

Trabajo Final

Bases de Datos 1

Jerónimo Hoyos Botero · Dylan Ramírez Blanquicet

Contenido

1	Esquema escogido para la implementación	2
2	Fase I. Implementación de esquemas	2
2.1	Creación de tablas en MySQL	2
2.2	Creación de tablas en PostgreSQL	3
2.3	Diseño de los experimentos	4
3	Fase II. Carga de datos	5
3.1	Código de población	5
3.2	Tiempos de inserción	6
4	Fase III. Consultas en SQL	6
4.1	Consulta 1	6
4.2	Consulta 2	8
5	Fase IV. Ejecución de las consultas en Python	9
5.1	Conexión y medición de tiempos	9
5.2	Solución de las consultas en Python	10
5.3	SQL contra Python	11
6	Resultados generales y conclusiones	12
A	Anexo. Capturas de la ejecución de los benchmarks	14
A.1	Fase II. Carga de datos	15
A.2	Fase III. Consultas en SQL	16
A.3	Fase IV. Ejecución desde Python	19

1 Esquema escogido para la implementación

De las relaciones elaboradas en el trabajo anterior se seleccionaron cuatro entidades conectadas entre sí. Estas entidades son **Especie**, **Pokémon**, **Entrenador** y **Tipo de Objeto**.

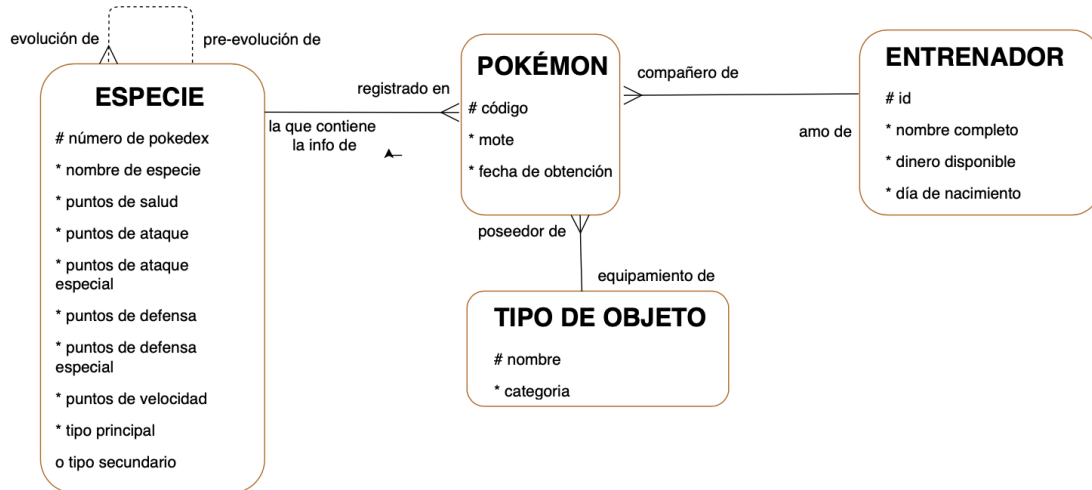


Figura 1. Modelo entidad-relación de las cuatro entidades seleccionadas.

2 Fase I. Implementación de esquemas

A partir del esquema entidad-relación se implementó la base de datos en MySQL y en PostgreSQL.

2.1 Creación de tablas en MySQL

```

1 CREATE TABLE especie (
2     numero_de_pokedex INT PRIMARY KEY,
3     nombre_especie VARCHAR(60) NOT NULL,
4     puntos_de_salud INT NOT NULL,
5     puntos_de_ataque INT NOT NULL,
6     puntos_de_ataque_esp INT NOT NULL,
7     puntos_de_defensa INT NOT NULL,
8     puntos_de_defensa_esp INT NOT NULL,
9     puntos_de_velocidad INT NOT NULL,
10    tipo_principal VARCHAR(20) NOT NULL,
11    tipo_secundario VARCHAR(20),
12    pre_evolution INT,
13    FOREIGN KEY (pre_evolution) REFERENCES especie (numero_de_pokedex)
14 );
  
```

Código 1. Tabla especie en MySQL.

```

1 CREATE TABLE entrenador (
  
```

```
2      id INT AUTO_INCREMENT PRIMARY KEY,  
3      nombre_completo VARCHAR(100) NOT NULL,  
4      dinero_disponible DECIMAL(10,2) NOT NULL,  
5      dia_de_nacimiento DATE NOT NULL  
6  );
```

Código 2. Tabla entrenador en MySQL.

```
1  CREATE TABLE tipo_de_objeto (  
2      nombre VARCHAR(50) PRIMARY KEY,  
3      categoria VARCHAR(50) NOT NULL  
4  );
```

Código 3. Tabla tipo_de_objeto en MySQL.

```
1  CREATE TABLE pokemon (  
2      codigo INT AUTO_INCREMENT PRIMARY KEY,  
3      mote VARCHAR(60) NOT NULL,  
4      fecha_de_obtencion DATE NOT NULL,  
5      numero_de_pokedex INT NOT NULL,  
6      id_entrenador INT NOT NULL,  
7      nombre_objeto VARCHAR(50),  
8      FOREIGN KEY (numero_de_pokedex) REFERENCES especie (numero_de_pokedex),  
9      FOREIGN KEY (id_entrenador) REFERENCES entrenador (id),  
10     FOREIGN KEY (nombre_objeto) REFERENCES tipo_de_objeto (nombre)  
11 );
```

Código 4. Tabla pokemon en MySQL.

2.2 Creación de tablas en PostgreSQL

```
1  CREATE TABLE especie (  
2      numero_de_pokedex INTEGER PRIMARY KEY,  
3      nombre_especie VARCHAR(60) NOT NULL,  
4      puntos_de_salud INTEGER NOT NULL,  
5      puntos_de_ataque INTEGER NOT NULL,  
6      puntos_de_ataque_esp INTEGER NOT NULL,  
7      puntos_de_defensa INTEGER NOT NULL,  
8      puntos_de_defensa_esp INTEGER NOT NULL,  
9      puntos_de_velocidad INTEGER NOT NULL,  
10     tipo_principal VARCHAR(20) NOT NULL,  
11     tipo_secundario VARCHAR(20),  
12     pre_evolucion INTEGER,  
13     FOREIGN KEY (pre_evolucion) REFERENCES especie (numero_de_pokedex)  
14 );
```

Código 5. Tabla especie en PostgreSQL.

```
1 CREATE TABLE entrenador (  
2     id SERIAL PRIMARY KEY,  
3     nombre_completo VARCHAR(100) NOT NULL,  
4     dinero_disponible NUMERIC(10,2) NOT NULL,  
5     dia_de_nacimiento DATE NOT NULL  
6 );
```

Código 6. Tabla entrenador en PostgreSQL.

```
1 CREATE TABLE tipo_de_objeto (  
2     nombre VARCHAR(50) PRIMARY KEY,  
3     categoria VARCHAR(50) NOT NULL  
4 );
```

Código 7. Tabla tipo_de_objeto en PostgreSQL.

```
1 CREATE TABLE pokemon (  
2     codigo SERIAL PRIMARY KEY,  
3     mote VARCHAR(60) NOT NULL,  
4     fecha_de_obtencion DATE NOT NULL,  
5     numero_de_pokedex INTEGER NOT NULL,  
6     id_entrenador INTEGER NOT NULL,  
7     nombre_objeto VARCHAR(50),  
8     FOREIGN KEY (numero_de_pokedex) REFERENCES especie (numero_de_pokedex),  
9     FOREIGN KEY (id_entrenador) REFERENCES entrenador (id),  
10    FOREIGN KEY (nombre_objeto) REFERENCES tipo_de_objeto (nombre)  
11 );
```

Código 8. Tabla pokemon en PostgreSQL.

2.3 Diseño de los experimentos

Todas las mediciones de tiempo reportadas en este trabajo se realizaron sobre un mismo equipo, de modo que los resultados fueran comparables entre sí.

COMPONENTE	ESPECIFICACIÓN
Procesador	Intel Core i5 de 4 núcleos
Gráficos	Intel Iris Plus Graphics 655
Memoria	8 GB LPDDR3

Tabla 1. Características del equipo empleado en los experimentos.

3 Fase II. Carga de datos

3.1 Código de población

Los datos se generaron con un script de Python. Para cada tamaño de carga el script crea un archivo .sql que inserta n filas y registra el tiempo de ejecución.

A continuación se muestra el bloque de carga de cada motor, donde los puntos suspensivos representan las n filas que se repiten en cada tabla.

```
1  SET @inicio = NOW(6);
2  START TRANSACTION;
3      INSERT INTO especie (numero_de_pokedex, nombre_especie, puntos_de_salud,
4      puntos_de_ataque, puntos_de_ataque_esp, puntos_de_defensa, puntos_de_defensa_esp,
5      puntos_de_velocidad, tipo_principal, tipo_secundario, pre_evolucion) VALUES (1,
6      'especie_1', 118, 74, 132, 96, 51, 160, 'fuego', 'agua', NULL);
7      ...
8      INSERT INTO entrenador (id, nombre_completo, dinero_disponible, dia_de_nacimiento)
9      VALUES (1, 'entrenador_1', '523847.19', '2011-04-23');
10     ...
11     INSERT INTO tipo_de_objeto (nombre, categoria) VALUES ('objeto_1', 'pocion');
12     ...
13     INSERT INTO pokemon (codigo, mote, fecha_de_obtencion, numero_de_pokedex,
14     id_entrenador, nombre_objeto) VALUES (1, 'mote_1', '2015-08-12', 5, 3, 'objeto_7');
15     ...
16 COMMIT;
17 SELECT TIMEDIFF(NOW(6), @inicio) AS tiempo_total_de_la_carga;
```

Código 9. Bloque de carga para MySQL.

```
1  DO $$
2  DECLARE
3      inicio TIMESTAMP := clock_timestamp();
4  BEGIN
5      INSERT INTO especie (numero_de_pokedex, nombre_especie, puntos_de_salud,
6      puntos_de_ataque, puntos_de_ataque_esp, puntos_de_defensa, puntos_de_defensa_esp,
7      puntos_de_velocidad, tipo_principal, tipo_secundario, pre_evolucion) VALUES (1,
8      'especie_1', 118, 74, 132, 96, 51, 160, 'fuego', 'agua', NULL);
9      ...
10     INSERT INTO entrenador (id, nombre_completo, dinero_disponible, dia_de_nacimiento)
11     VALUES (1, 'entrenador_1', '523847.19', '2011-04-23');
12     ...
13     INSERT INTO tipo_de_objeto (nombre, categoria) VALUES ('objeto_1', 'pocion');
14     ...
15     INSERT INTO pokemon (codigo, mote, fecha_de_obtencion, numero_de_pokedex,
16     id_entrenador, nombre_objeto) VALUES (1, 'mote_1', '2015-08-12', 5, 3, 'objeto_7');
17     ...
18     RAISE NOTICE 'Tiempo total de la carga: %', clock_timestamp() - inicio;
19 END $$;
```

Código 10. Bloque de carga para PostgreSQL.

3.2 Tiempos de inserción

Se realizaron tres pruebas de inserción en cada SGBD, con 1.000, 10.000 y 100.000 filas por tabla, y se midió el tiempo empleado en cada caso.

PRUEBA DE CARGA	MYSQL	POSTGRESQL
1.000 filas	0,370492	0,226006
10.000 filas	3,596290	2,252994
100.000 filas	35,933864	25,74875

Tabla 2. Tiempo de inserción por prueba de carga, en segundos.

En el siguiente gráfico se aprecia la comparación de estos tiempos entre ambos motores.

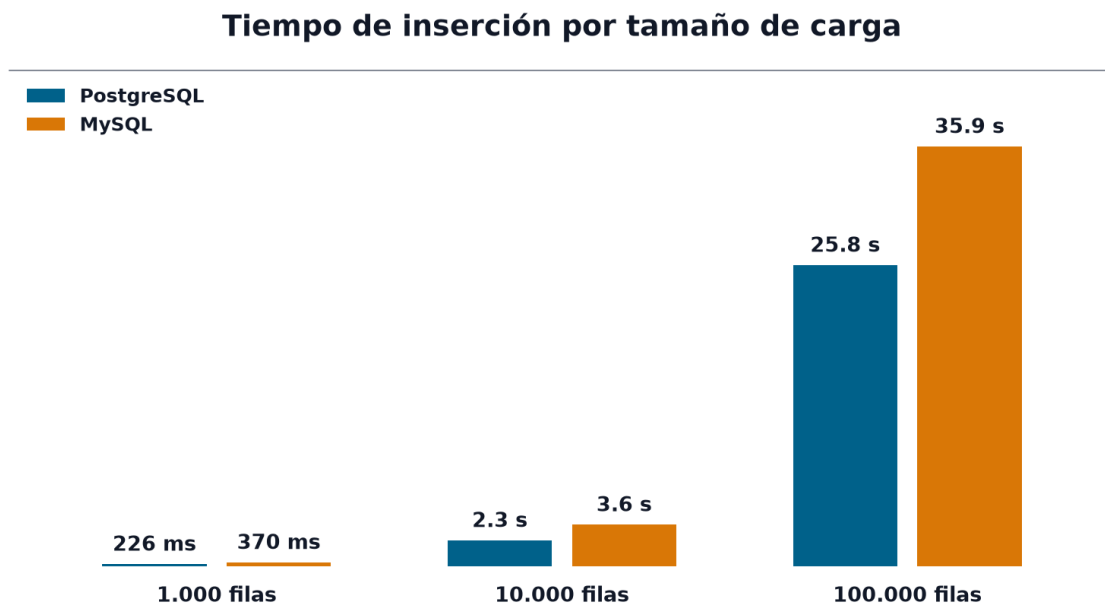


Figura 2. Tiempo de inserción por tamaño de carga en ambos motores.

En las tres pruebas de inserción el motor PostgreSQL resultó significativamente más rápido, entre 1,4 y 1,6 veces por encima de MySQL, y la ventaja se mantuvo en todos los tamaños de carga.

4 Fase III. Consultas en SQL

Las dos consultas son idénticas en MySQL y en PostgreSQL, por lo que se muestra una sola versión de cada una.

4.1 Consulta 1

Devuelve los pokémon cuyo tipo principal es fuego, junto con el nombre y la velocidad de su especie y el nombre del entrenador que los posee. Para ello combina las tablas pokemon, especie y entrenador mediante JOIN.

```
1  SELECT pokemon.mote,
2         especie.nombre_especie,
3         especie.puntos_de_velocidad,
4         entrenador.nombre_completo
5  FROM pokemon
6  JOIN especie ON pokemon.numero_de_pokedex = especie.numero_de_pokedex
7  JOIN entrenador ON pokemon.id_entrenador = entrenador.id
8  WHERE especie.tipo_principal = 'fuego';
```

Código 11. Consulta 1.

En la siguiente tabla se muestran las primeras cinco filas del resultado.

MOTE	NOMBRE_ESPECIE	PUNTOS_DE_VELOCIDAD	NOMBRE_COMPLETO
mote_39	especie_73737	147	entrenador_90283
mote_65	especie_68064	107	entrenador_1986
mote_70	especie_35863	39	entrenador_68181
mote_71	especie_69590	88	entrenador_27336
mote_78	especie_81380	54	entrenador_61988

Tabla 3. Primeras cinco filas de la consulta 1 sobre la carga de 100.000 filas.

En el siguiente gráfico se presentan los tiempos de la consulta.

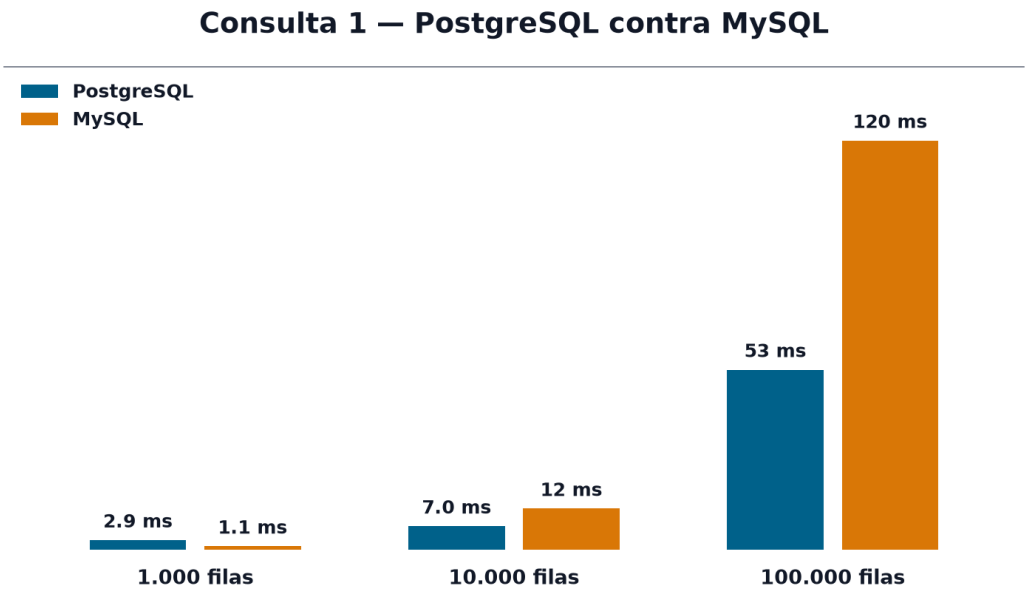


Figura 3. PostgreSQL contra MySQL en la Consulta 1.

En la primera carga MySQL fue superior, con 1,12 ms contra 2,93 ms, pero en la segunda y en la tercera PostgreSQL remontó. En la última la diferencia a su favor resultó más evidente, 52,8 ms contra 120 ms, es decir, más del doble de rapidez.

4.2 Consulta 2

Cuenta cuántos pokémon tiene cada entrenador según la categoría del objeto que llevan equipado. Agrupa por entrenador y categoría, y ordena el resultado de mayor a menor cantidad, resolviendo los empates por el nombre del entrenador.

```
1 SELECT entrenador.nombre_completo,
2     tipo_de_objeto.categoria,
3     COUNT(*) AS total_pokemon
4 FROM pokemon
5 JOIN entrenador ON pokemon.id_entrenador = entrenador.id
6 JOIN tipo_de_objeto ON pokemon.nombre_objeto = tipo_de_objeto.nombre
7 GROUP BY entrenador.nombre_completo, tipo_de_objeto.categoria
8 ORDER BY total_pokemon DESC, entrenador.nombre_completo;
```

Código 12. Consulta 2.

En la siguiente tabla se muestran las primeras cinco filas del resultado.

NOMBRE_COMPLETO	CATEGORIA	TOTAL_POKEMON
entrenador_66533	piedra	4
entrenador_11979	llave	3
entrenador_12463	pocion	3
entrenador_12501	tesoro	3
entrenador_13494	tesoro	3

Tabla 4. Primeras cinco filas de la consulta 2 sobre la carga de 100.000 filas.

En el siguiente gráfico se muestra una comparación de los tiempos de la consulta.

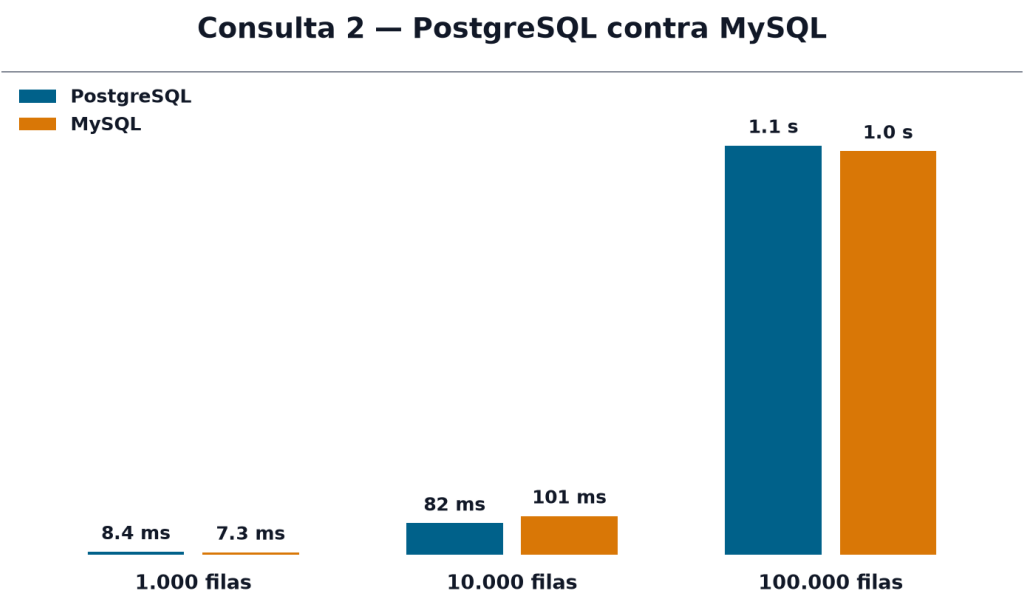


Figura 4. PostgreSQL contra MySQL en la Consulta 2.

Aquí los dos motores quedaron mucho más parejos que en la consulta anterior. MySQL fue más rápido con 1.000 filas, 7,28 ms contra 8,39 ms, y también con 100.000 filas, donde marcó 1044 ms contra 1058 ms, apenas un 1,4 % de diferencia. Con 10.000 filas, en cambio, el más rápido fue PostgreSQL, con 82,3 ms contra 101 ms. No se encontró una clara ventaja entre motores.

5 Fase IV. Ejecución de las consultas en Python

Desde Python se abre una conexión propia con el SGBD, se traen mediante **SELECT * FROM tabla** únicamente las tablas necesarias y se combinan los datos con código propio donde el cronómetro cubre todo ese recorrido.

5.1 Conexión y medición de tiempos

El código de conectividad y de medición es idéntico para MySQL y PostgreSQL. Lo único que cambia entre ambos es la cadena de conexión.

```
1 # MySQL
2 CONEXION = "mysql+pymysql://root:@localhost:3306/{base}"
3 # PostgreSQL
4 CONEXION = "postgresql+psycopg2://postgres:1234@localhost:5432/{base}"
```

Código 13. Comparación de las conexiones entre los SGBD.

El resto del código es común a ambos motores. Las funciones `consulta_1` y `consulta_2` se muestran en la subsección siguiente.

```
1 import os
2 import time
3 import pandas as pd
4 from sqlalchemy import create_engine
5
6 BASE = os.path.dirname(os.path.abspath(__file__))
7 TABLAS = ["especie", "entrenador", "tipo_de_objeto", "pokemon"]
8 BASES_DE_DATOS = ["pokemon_1k", "pokemon_10k", "pokemon_100k"]
9 CONEXION = "Depende del motor"
10
11 def traer_tablas(url, tablas=TABLAS):
12     with create_engine(url).connect() as conexion:
13         return {tabla: pd.read_sql(f"SELECT * FROM {tabla}", conexion)
14                 for tabla in tablas}
15
16
17 def cronometrar(funcion, argumento):
18     inicio = time.perf_counter()
19     resultado = funcion(argumento)
20     return resultado, (time.perf_counter() - inicio) * 1000
21
22
23 medidas = []
```

```

24
25 for nombre_base in BASES_DE_DATOS:
26     url = CONEXION.format(base=nombre_base)
27     tablas, tiempo_traida = cronometrar(trazer_tablas, url)
28     resultado_1, tiempo_1 = cronometrar(consulta_1, url)
29     resultado_2, tiempo_2 = cronometrar(consulta_2, url)
30
31     filas_traidas = sum(len(tabla) for tabla in tablas.values())
32     medidas.append([nombre_base, "traer tablas", tiempo_traida, filas_traidas])
33     medidas.append([nombre_base, "consulta 1", tiempo_1, len(resultado_1)])
34     medidas.append([nombre_base, "consulta 2", tiempo_2, len(resultado_2)])
35
36 tiempos = pd.DataFrame(medidas,
37                         columns=["carga", "paso", "milisegundos", "filas"])
38 tiempos.to_csv(os.path.join(BASE, "tiempos.csv"), index=False)

```

Código 14. Conectividad y medición de tiempos, común a ambos motores.

5.2 Solución de las consultas en Python

Para las consultas en Python se empleó pandas, una de las librerías más usadas en ciencia de datos para el procesamiento de información.

```

1 def consulta_1(url):
2     tablas = traer_tablas(url, ["pokemon", "especie", "entrenador"])
3     con_especie = tablas["pokemon"].merge(tablas["especie"],
4                                           on="numero_de_pokedex")
5     con_entrenador = con_especie.merge(tablas["entrenador"],
6                                       left_on="id_entrenador", right_on="id")
7     de_fuego = con_entrenador[con_entrenador["tipo_principal"] == "fuego"]
8     return de_fuego[["mote", "nombre_especie",
9                     "puntos_de_velocidad", "nombre_completo"]]
10
11
12 def consulta_2(url):
13     tablas = traer_tablas(url, ["pokemon", "entrenador", "tipo_de_objeto"])
14     con_entrenador = tablas["pokemon"].merge(tablas["entrenador"],
15                                             left_on="id_entrenador", right_on="id")
16     con_objeto = con_entrenador.merge(tablas["tipo_de_objeto"],
17                                     left_on="nombre_objeto", right_on="nombre")
18     conteo = con_objeto.groupby(["nombre_completo", "categoria"]).size()
19     conteo = conteo.reset_index(name="total_pokemon")
20     return conteo.sort_values(["total_pokemon", "nombre_completo"],
21                              ascending=[False, True])
21

```

Código 15. Resolución de las consultas en Python.

5.3 SQL contra Python

En las siguientes gráficas se compara, para la consulta 1, el tiempo de MySQL frente al de la librería pandas.

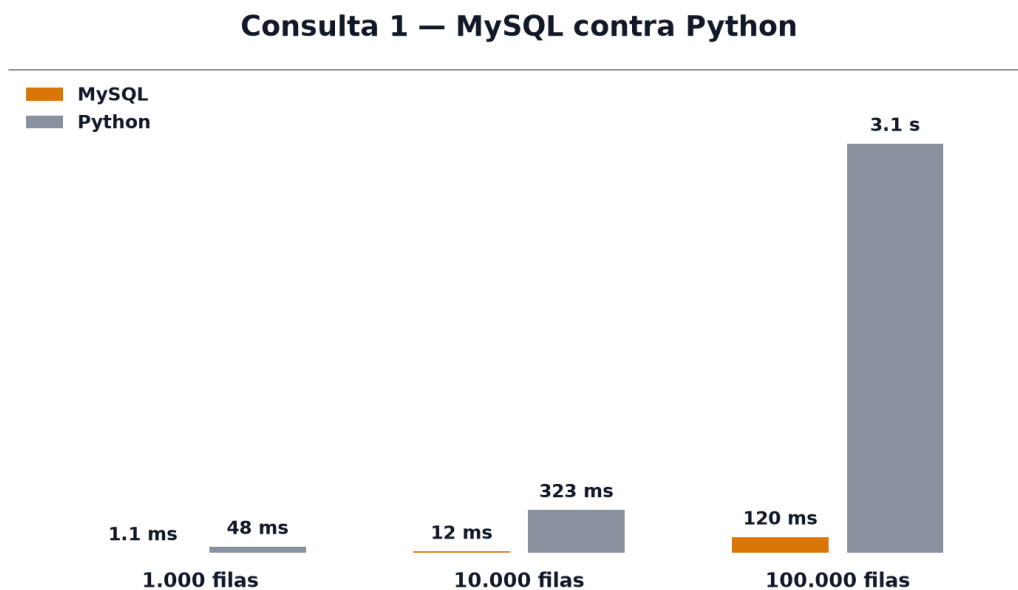


Figura 5. Consulta 1, MySQL frente a Python.

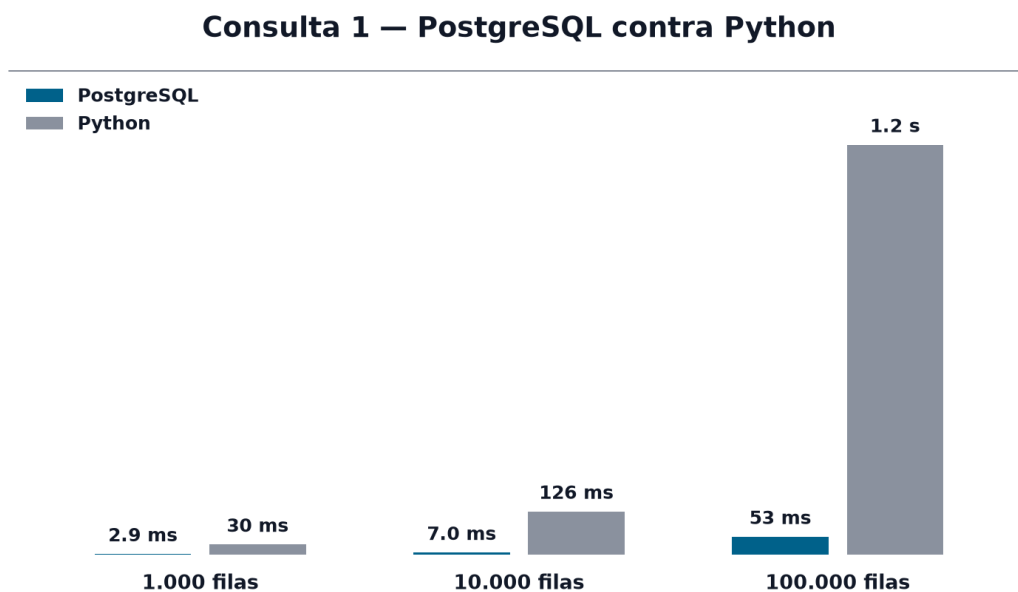


Figura 6. Consulta 1, PostgreSQL frente a Python.

En estas dos comparaciones los gestores de bases de datos superaron a la librería especializada de Python por un amplio margen.

En las siguientes gráficas se repite la comparación anterior, ahora con la consulta 2.

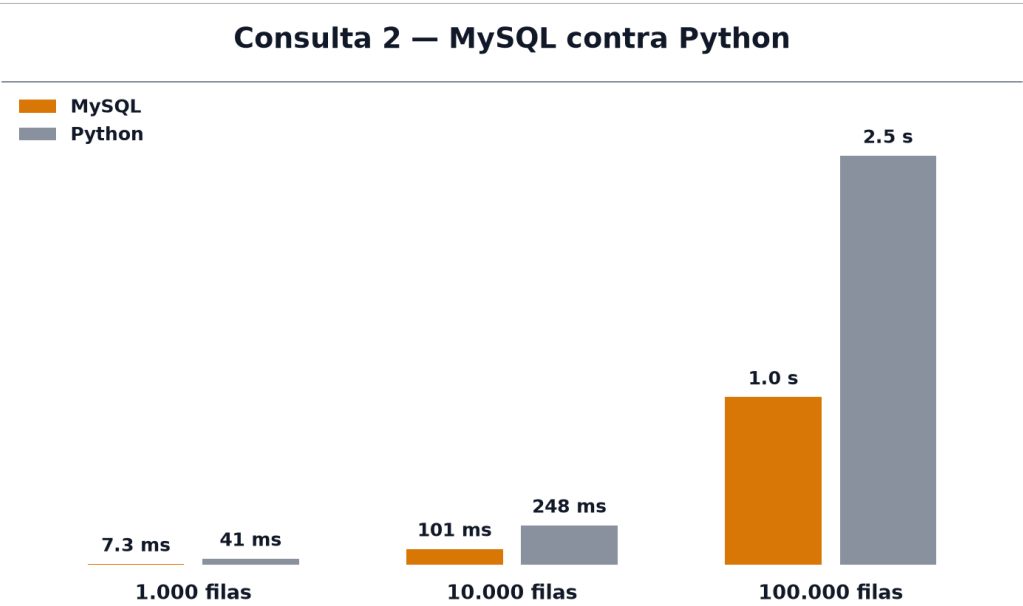


Figura 7. Consulta 2, MySQL frente a Python.

En el caso de MySQL se mantuvo la superioridad, aunque con una diferencia menor que en la consulta 1.

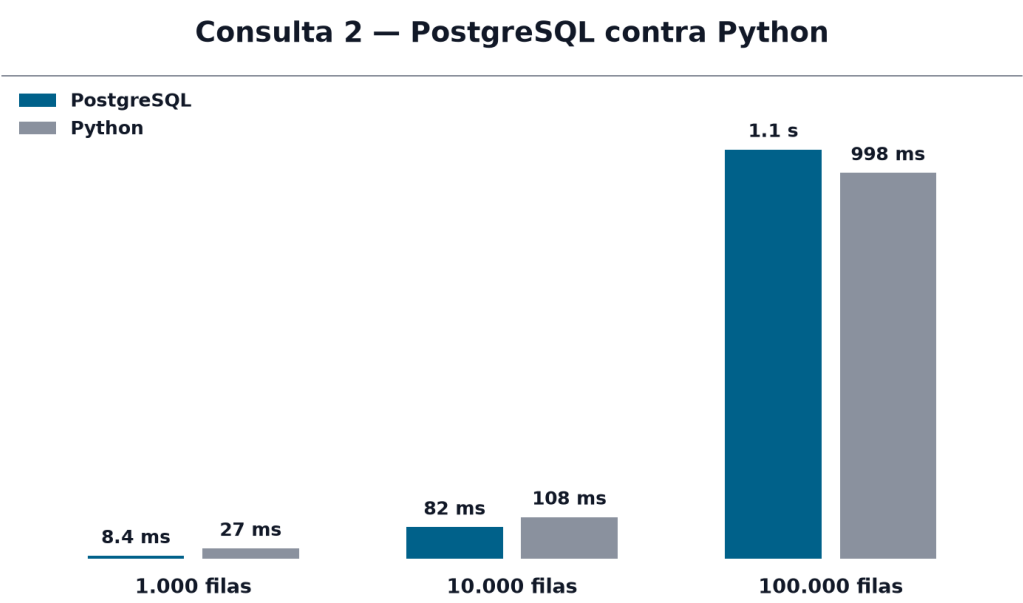


Figura 8. Consulta 2, PostgreSQL frente a Python.

A PostgreSQL, en cambio, le costó bastante más. Conservó la ventaja en las dos primeras cargas, con 8,39 ms contra 27,4 ms y con 82,3 ms contra 108 ms, pero el margen se fue estrechando y en la carga de 100.000 filas pandas terminó por delante, 998 ms contra 1058 ms. Es el único escenario de todo el trabajo en el que la librería de Python superó a un motor de bases de datos.

6 Resultados generales y conclusiones

En la siguiente tabla se muestran todos los resultados de las consultas.

CONSULTA	CARGA	MYSQL	POSTGRESQL	PYTHON DESDE MYSQL	PYTHON DESDE PG
Consulta 1	1.000	1,120	2,928	48,183	30,413
	10.000	12,144	6,994	323,382	125,677
	100.000	119,870	52,796	3064,940	1184,717
Consulta 2	1.000	7,281	8,393	40,885	27,445
	10.000	100,983	82,338	247,794	108,340
	100.000	1043,776	1058,145	2543,436	997,931

Tabla 5. Tiempos de respuesta de las consultas por escenario y carga, en milisegundos.

De estos resultados se concluye que en la consulta 1 el mejor escenario fue PostgreSQL, que ganó en las cargas de 10.000 y de 100.000 filas, y el peor la llamada de Python desde MySQL.

En la consulta 2, en cambio, los dos motores quedaron prácticamente empatados, y el mejor tiempo de la carga más grande no correspondió a ninguno de ellos sino a Python desde PostgreSQL, con 998 ms. El peor escenario volvió a ser Python desde MySQL en las tres cargas.

En la siguiente gráfica se muestran ambas consultas en escala logarítmica.

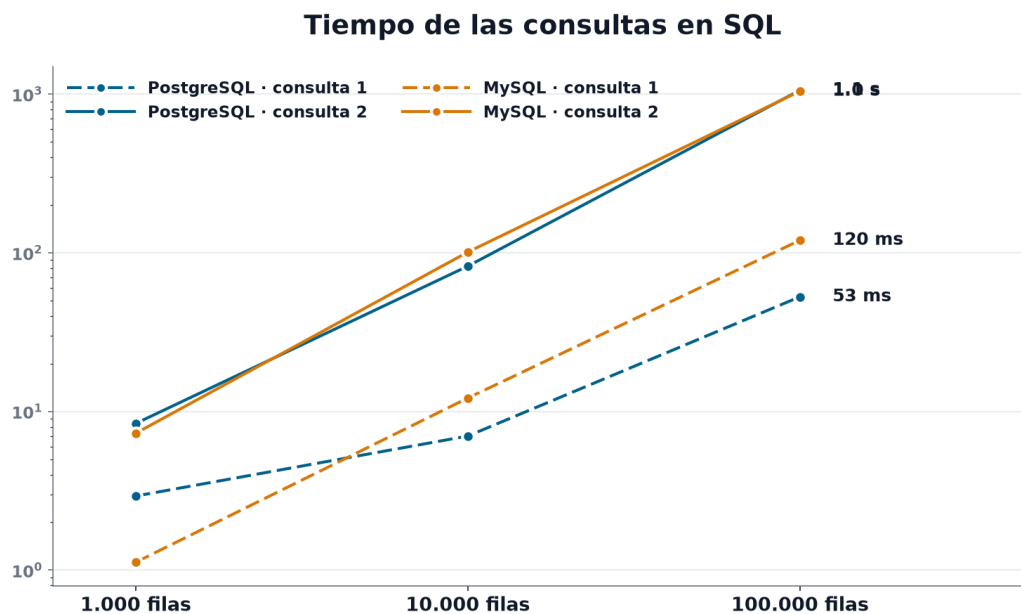


Figura 9. Tiempo de las consultas en SQL.

Se observa que la diferencia entre motores es más marcada en la consulta 1, donde a partir de las 10.000 filas la curva de PostgreSQL queda claramente por debajo de la de MySQL, mientras que en la consulta 2 las dos curvas quedan casi superpuestas. Se aprecia también que la consulta 2 es aproximadamente un orden de magnitud más costosa que la consulta 1 en la carga de 100.000 filas, lo que se explica por el GROUP BY y el ORDER BY que debe resolver.

Finalmente, en el siguiente gráfico se muestra la suma de los tiempos de ambas consultas.

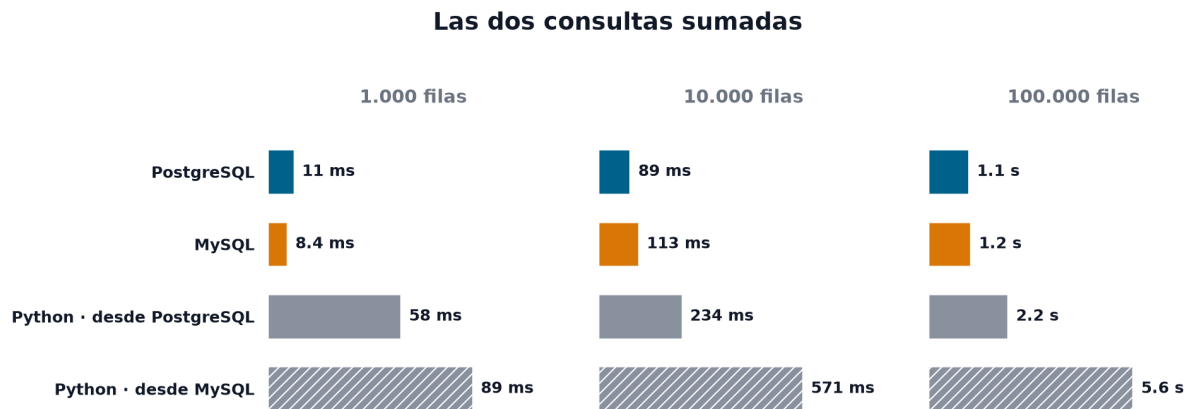


Figura 10. Las dos consultas sumadas, por motor y por carga.

Se aprecia que con 1.000 filas el mejor es MySQL, con 8,4 ms contra 11 ms. Al crecer la carga PostgreSQL pasa al frente, primero con holgura, 89 ms contra 113 ms, y después por muy poco, 1,11 s contra 1,16 s. El peor resultado corresponde, en las tres cargas, a la llamada de MySQL desde Python.

En conclusión, se pueden destacar los siguientes puntos.

- En el uso exclusivo de JOIN fue superior PostgreSQL, con más del doble de rapidez que MySQL en la carga de 100.000 filas.
- En las operaciones GROUP BY y ORDER BY (consulta 2) los dos motores quedaron parejos. Por lo tanto no se podría quien fue superior.
- Las llamadas desde Python junto con el uso de librerías especializadas no resultaron óptimas, y fue preferible ejecutar un script directamente dentro del SGBD. La única excepción fue la consulta 2 sobre 100.000 filas leída desde PostgreSQL, donde pandas quedó por delante del motor por un margen estrecho.
- Para la consulta de datos se mostró una ventaja general de los sistemas de gestión de bases de datos frente a los scripts de Python.
- En cuanto a la inserción, PostgreSQL fue significativamente mejor que MySQL en los tres tamaños de carga.

A Anexo. Capturas de la ejecución de los benchmarks

Las capturas de pantalla que siguen son el respaldo de los tiempos citados en el trabajo. Corresponden a la ejecución del código tal como la devolvieron el cliente de cada motor y el intérprete de Python.

A.1 Fase II. Carga de datos

```
Dylan@MacBook-Pro-de-Manuela ~ % mysql -u root pokemon_1k < /Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/mysql/carga_de_datos/carga_1k.sql
tiempo_total_de_la_carga
00:00:00.370492
```

Figura 11. MySQL, carga de 1.000 filas por tabla.

```
Dylan@MacBook-Pro-de-Manuela ~ % mysql -u root pokemon_10k < /Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/mysql/carga_de_datos/carga_10k.sql
tiempo_total_de_la_carga
00:00:03.596290
```

Figura 12. MySQL, carga de 10.000 filas por tabla.

```
Dylan@MacBook-Pro-de-Manuela ~ % mysql -u root pokemon_100k < /Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/mysql/carga_de_datos/carga_100k.sql
tiempo_total_de_la_carga
00:00:35.933864
```

Figura 13. MySQL, carga de 100.000 filas por tabla.

```
Dylan@MacBook-Pro-de-Manuela ~ % psql -U postgres -d pokemon_1k -f /Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/postgreSQL/carga_de_datos/carga_1k.sql
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
psql:/Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/postgreSQL/carga_de_datos/carga_1k.sql:4050: NOTICE: Tiempo total de la carga: 00:00:00.226006
DO
Dylan@MacBook-Pro-de-Manuela ~ %
```

Figura 14. PostgreSQL, carga de 1.000 filas por tabla.

```
Dylan@MacBook-Pro-de-Manuela ~ % psql -U postgres -d pokemon_10k -f /Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/postgreSQL/carga_de_datos/carga_10k.sql
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
psql:/Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/postgreSQL/carga_de_datos/carga_10k.sql:40050: NOTICE: Tiempo total de la carga: 00:00:02.252994
DO
Dylan@MacBook-Pro-de-Manuela ~ %
```

Figura 15. PostgreSQL, carga de 10.000 filas por tabla.

```
Dylan@MacBook-Pro-de-Manuela ~ % psql -U postgres -d pokemon_100k -f /Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/postgreSQL/carga_de_datos/carga_100k.sql
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
psql:/Users/Dylan/Proyectos/pokemon/mysql-vs-postgres-benchmark/postgreSQL/carga_de_datos/carga_100k.sql:400050: NOTICE: Tiempo total de la carga: 00:00:25.74875
DO
```

Figura 16. PostgreSQL, carga de 100.000 filas por tabla.

A.2 Fase III. Consultas en SQL

```
tiempo_consulta_1
00:00:00.001120
Dylan@MacBook-Pro-de-Manuela ~ %
```

Figura 17. MySQL, consulta 1 sobre 1.000 filas.

```
tiempo_consulta_1
00:00:00.012144
```

Figura 18. MySQL, consulta 1 sobre 10.000 filas.

```
tiempo_consulta_1
00:00:00.119870
Dylan@MacBook-Pro-de-Manuela ~ %
```

Figura 19. MySQL, consulta 1 sobre 100.000 filas.

```
tiempo_consulta_2
00:00:00.007281
Dylan@MacBook-Pro-de-Manuela ~ %
```

Figura 20. MySQL, consulta 2 sobre 1.000 filas.

```
tiempo_consulta_2
00:00:00.100983
Dylan@MacBook-Pro-de-Manuela ~ %
```

Figura 21. MySQL, consulta 2 sobre 10.000 filas.

```
tiempo_consulta_2
00:00:01.043776
Dylan@MacBook-Pro-de-Manuela ~ %
```

Figura 22. MySQL, consulta 2 sobre 100.000 filas.

```
mote_573 | especie_121 |
mote_620 | especie_69  |
Time: 2,928 ms
Dylan@MacBook-Pro-de-Manuela ~ %
```

Figura 23. PostgreSQL, consulta 1 sobre 1.000 filas.

```
mote_317 | especie_6622 |
mote_339 | especie_4120 |
mote_358 | especie_9550 |
mote_373 | especie_2878 |
mote_401 | especie_1612 |
Time: 6,994 ms
```

Figura 24. PostgreSQL, consulta 1 sobre 10.000 filas.

```
mote_327 | especie_87399 |  
mote_340 | especie_54291 |  
mote_343 | especie_84693 |  
mote_376 | especie_68441 |  
mote_378 | especie_31707 |  
mote_429 | especie_22899 |  
mote_461 | especie_33253 |  
mote_478 | especie_62940 |  
Time: 52,796 ms
```

Figura 25. PostgreSQL, consulta 1 sobre 100.000 filas.

```
entrenador_560 | llave  
entrenador_576 | llave  
entrenador_590 | tecnica  
entrenador_644 | baya  
Time: 8,393 ms
```

Figura 26. PostgreSQL, consulta 2 sobre 1.000 filas.

```
entrenador_1208 | tecnica |  
entrenador_1222 | tesoro |  
entrenador_1227 | baya |  
entrenador_1229 | baya |  
entrenador_1288 | pocion |  
Time: 82,338 ms
```

Figura 27. PostgreSQL, consulta 2 sobre 10.000 filas.

```
entrenador_23478 | tesoro |  
entrenador_23538 | piedra |  
entrenador_25206 | tecnica |  
entrenador_25641 | llave |  
entrenador_25908 | baya |  
Time: 1058,145 ms (00:01,058)
```

Figura 28. PostgreSQL, consulta 2 sobre 100.000 filas.

A.3 Fase IV. Ejecución desde Python

```
(venv) Dylan@MacBook-Pro-de-Manuela python % python3 consultas.py
pokemon_1k: traer 82.083 ms, consulta 1 48.183 ms, consulta 2 40.885 ms
pokemon_10k: traer 364.944 ms, consulta 1 323.382 ms, consulta 2 247.794 ms
pokemon_100k: traer 3595.773 ms, consulta 1 3064.940 ms, consulta 2 2543.436 ms
```

	carga	paso	milisegundos	filas
0	pokemon_1k	traer tablas	82.082722	4000
1	pokemon_1k	consulta 1	48.183232	38
2	pokemon_1k	consulta 2	40.884735	674
3	pokemon_10k	traer tablas	364.943748	40000
4	pokemon_10k	consulta 1	323.381953	594
5	pokemon_10k	consulta 2	247.793814	6617
6	pokemon_100k	traer tablas	3595.772771	400000
7	pokemon_100k	consulta 1	3064.939648	5390
8	pokemon_100k	consulta 2	2543.436109	66580

```
(venv) Dylan@MacBook-Pro-de-Manuela python %
```

Figura 29. Ejecución del script de Python contra MySQL.

```
(venv) Dylan@MacBook-Pro-de-Manuela python % python3 consultas.py
pokemon_1k: traer 95.952 ms, consulta 1 30.413 ms, consulta 2 27.445 ms
pokemon_10k: traer 152.862 ms, consulta 1 125.677 ms, consulta 2 108.340 ms
pokemon_100k: traer 1222.129 ms, consulta 1 1184.717 ms, consulta 2 997.931 ms
```

	carga	paso	milisegundos	filas
0	pokemon_1k	traer tablas	95.952069	4000
1	pokemon_1k	consulta 1	30.412820	38
2	pokemon_1k	consulta 2	27.444831	674
3	pokemon_10k	traer tablas	152.862016	40000
4	pokemon_10k	consulta 1	125.676557	594
5	pokemon_10k	consulta 2	108.340156	6617
6	pokemon_100k	traer tablas	1222.129147	400000
7	pokemon_100k	consulta 1	1184.716555	5390
8	pokemon_100k	consulta 2	997.930683	66580

```
(venv) Dylan@MacBook-Pro-de-Manuela python %
```

Figura 30. Ejecución del script de Python contra PostgreSQL.